

Extending Kubernetes Autoscaling with a Serverless AWS IoT Pipeline

Julia Meneses Roberto
PSI5120 - Cloud Computing Topics, 2026

Abstract- This paper extends a previously validated Kubernetes Horizontal Pod Autoscaler project with an end-to-end, software-only Internet of Things pipeline on Amazon Web Services. A Python device simulator authenticates to AWS IoT Core using X.509 mutual TLS, publishes temperature and humidity telemetry over MQTT at QoS 1, receives cloud-to-device commands, and synchronizes actuator state through Device Shadow. An IoT SQL rule invokes AWS Lambda, which validates and persists readings in Amazon DynamoDB. The existing FastAPI service is expanded with telemetry, command, and Shadow endpoints while retaining its CPU-based HPA. A live experiment produced three MQTT readings, three successful Lambda writes, Shadow convergence from desired to reported temperature 15, successful command delivery, and a controlled authorization denial. The extension preserves the intermediate submission as an immutable Git tag and demonstrates how portable container autoscaling can coexist with managed event-driven cloud services.

Index Terms- AWS IoT Core, MQTT, Kubernetes, Horizontal Pod Autoscaler, Lambda, DynamoDB, Device Shadow, FastAPI.

I. INTRODUCTION

Cloud applications increasingly combine request-driven services with asynchronous device telemetry. Kubernetes is effective at scaling stateless HTTP workloads, while managed IoT and serverless services provide device identity, message routing, event processing, and durable storage. Treating either model in isolation hides important boundaries involving trust, state, operations, and cost.

The intermediate project compared the same CPU-bound FastAPI workload under the Kubernetes Horizontal Pod Autoscaler (HPA) on Minikube and Amazon EKS. Six experiments showed expansion from one to six pods and recovery to one. This final project follows option 2.2 by extending that working system instead of replacing it. The original version is frozen at Git tag *ta1-submission*.

The extension uses no physical device. A container-ready Python program simulates a temperature sensor and fan actuator. This choice retains the networking, authentication, authorization, event, and state-management properties of an IoT solution while making the experiment reproducible without laboratory hardware.

II. OBJECTIVES AND RESEARCH QUESTIONS

RQ1 asks whether a least-privilege software device can complete bidirectional MQTT communication with AWS IoT Core. RQ2 asks whether rule-based, serverless ingestion reliably persists each telemetry reading. RQ3 asks whether Device Shadow can reconcile a desired actuator target with the device-reported state. RQ4 asks what changes when an autoscaled Kubernetes API is integrated with managed IoT services while retaining the original HPA.

The acceptance criteria were: X.509 mTLS connection; three QoS 1 publishes; command cmd-001 delivered; three Lambda invocations and DynamoDB items; desired and reported target temperature equal to 15; denied subscription outside the policy; eight passing API tests; and valid Kustomize output for the final application image.

III. BACKGROUND

A. MQTT and mutual TLS

MQTT is a lightweight publish/subscribe protocol organized around topics. QoS 1 provides at-least-once delivery and therefore permits duplicates; consumers should use stable identifiers or idempotent writes. AWS IoT Core authenticates the simulated device with an X.509 certificate during mutual TLS. Authentication proves identity, whereas the attached IoT policy independently authorizes connect, publish, subscribe, and receive operations.

B. AWS IoT rules and serverless processing

An AWS IoT SQL rule filters messages from the device telemetry topic and invokes Lambda. This decouples the broker from persistence and avoids embedding database credentials in the simulator. Lambda receives the decoded JSON document, validates required fields, converts numeric values to DynamoDB-compatible decimal types, and performs PutItem using a role restricted to the named table plus CloudWatch Logs.

C. Device Shadow

Device Shadow maintains a cloud-side document with desired, reported, and delta state. The API or operator writes a desired target. AWS publishes the delta to a reserved MQTT topic when reported state differs. The simulator applies the target and publishes reported state from an auxiliary thread, avoiding a deadlock in the MQTT SDK callback thread.

D. Kubernetes HPA boundary

HPA remains responsible for the FastAPI replica count, not the IoT broker, Lambda concurrency, or DynamoDB capacity. The Deployment requests 100 millicores per pod, targets 50 percent average CPU, and allows one to six replicas. The extension therefore combines two elasticity models: a Kubernetes control loop for HTTP compute and provider-managed scaling for event ingestion.

IV. SYSTEM ARCHITECTURE

Layer	Technology	Responsibility
Device	Python + AWS IoT SDK	Sensor, actuator, mTLS
Messaging	AWS IoT Core	MQTT routing and Shadow
Processing	IoT Rule + Lambda	filter and validation
Storage	DynamoDB	telemetry history
Service	FastAPI + HPA	queries and commands

The telemetry path is simulator to IoT Core to IoT Rule to Lambda to DynamoDB. The control path is FastAPI or AWS data plane to the device-specific command topic. The state path uses Device Shadow desired, delta, and reported documents. CloudWatch captures Lambda execution evidence. Resource names include the jmr identifier and all resources reside in us-east-1.

A. Data model

DynamoDB uses device_id as the partition key and sequence as the numeric sort key. This supports ordered queries for a single device and turns repeated delivery of the same sequence into replacement rather than unbounded duplication. Additional attributes are timestamp, temperature, humidity, and actuator state.

B. API extension

FastAPI version 2.0 adds GET /telemetry, POST /commands, GET /device-shadow, and POST /device-shadow/desired. The AWS SDK follows the standard credential chain, so no credentials are built into the image. Environment variables contain only region and resource names. Decimal values are converted before JSON serialization.

V. SECURITY DESIGN

A. Device least privilege

The IoT policy permits the certificate to connect only with the expected client identifier. Publish is limited to the device telemetry topic and its Shadow update topic. Subscribe and receive are limited to the command and Shadow delta topics. A wildcard forbidden topic is deliberately absent, providing a negative test of authorization.

B. Serverless least privilege

The Lambda trust policy names only `lambda.amazonaws.com`. Its inline role permits CloudWatch log creation and DynamoDB PutItem only on the experiment table. The IoT service receives Lambda invocation permission constrained by the rule ARN. The table uses on-demand billing, eliminating provisioned idle throughput for this small experiment.

C. Secret hygiene

The certificate and private key were generated in AWS CloudShell after explicit user authorization. The key remained in a non-versioned certs directory with restrictive permissions. Git ignores PEM, CRT, and KEY files. Evidence omits the AWS account identifier, endpoint, certificate ARN, credentials, request identifiers, and all key material.

D. Container controls

Both the API and simulator images run with UID 10001. The API Deployment disables privilege escalation, drops Linux capabilities, uses a read-only root filesystem, declares CPU and memory requests and limits, and includes readiness and liveness probes. Dependencies are pinned to make rebuilds auditable.

VI. IMPLEMENTATION

The simulator establishes MQTT over mutual TLS, subscribes at QoS 1, reports an online Shadow state, and emits deterministic readings with sequences beginning at 1001. A command callback updates the in-memory actuator. A Shadow callback copies desired fields to reported state from a worker thread. On termination, the client reports offline state and disconnects cleanly.

The Lambda handler rejects events missing `device_id`, `sequence`, `temperature_c`, or `timestamp_utc`. Accepted numbers are converted through strings into Decimal objects, preventing floating-point serialization errors. A structured `dynamodb_put_ok` message records the device and sequence without exposing message contents or credentials.

Provisioning is automated in a commented CloudShell script. It creates the table, IAM role, function package, invocation permission, IoT rule, Thing, policy, and active certificate. A companion cleanup script enumerates principals, detaches policy and Thing associations, deactivates and deletes certificates, and removes named cloud resources only after evidence review.

VII. METHODOLOGY

The local stage executed eight automated endpoint tests with AWS clients replaced by test doubles. Python compilation and Kustomize rendering were checked independently. The live stage provisioned the named resources in `us-east-1` and ran the simulator in CloudShell using its temporary console credentials and locally held certificate files.

Three readings were emitted at three-second intervals. While the device remained connected, an IoT data-plane publish sent command `cmd-001` with value `fan-on`. A Shadow update set `target_temperature` to 15. After disconnection, the negative mode authenticated successfully and attempted an unauthorized wildcard subscription.

DynamoDB, CloudWatch Logs, and the final Shadow document were then queried.

A. Controlled variables

The Thing name, MQTT client identifier, region, telemetry topic, command topic, certificate, QoS, Lambda version, table schema, and publishing interval were held constant. Sequence numbers and generated sensor values were deterministic. This makes the evidence traceable across simulator output, Lambda logs, and DynamoDB records without depending on personal data or random payloads.

B. Evidence strategy

Each acceptance criterion was corroborated at the component that owns it. Device logs establish connection, publish acknowledgments, commands, and Shadow deltas. CloudWatch establishes Lambda execution. A DynamoDB query establishes durable state. The final Shadow document establishes convergence. The forbidden subscription establishes policy enforcement. No conclusion relies exclusively on a success message produced by the component under test.

C. Safety and cleanup sequencing

Evidence was collected before teardown because certificate deletion and table removal are irreversible. Screens and text were reviewed for secrets before repository inclusion. The private key, certificate body, endpoint, account number, and ARN were never copied into the paper. Cleanup remains a deliberate final step after the PDF and tagged commit are verified.

D. HPA continuity method

The final Kubernetes overlay is a narrow patch over the validated base. It changes the application image tag to 2.0.0 and adds only non-secret AWS resource identifiers. Namespace, Service, resource requests, probes, security context, HPA target, replica bounds, and scaling behavior remain inherited from TA1. This minimizes confounding changes and makes the extension auditable as a versioned delta.

VIII. RESULTS

Criterion	Observed result	Status
mTLS	connection accepted	Pass
Telemetry	sequences 1001-1003	Pass
Command	cmd-001 fan-on received	Pass
Lambda	3 dynamodb_put_ok logs	Pass
DynamoDB	Count=3	Pass
Shadow	desired=reported=15	Pass
Forbidden topic	subscribe denied	Pass
API tests	8 passed	Pass

A. Telemetry and persistence

The device published temperatures 21.8, 22.1, and 22.4 degrees Celsius for sequences 1001, 1002, and 1003. DynamoDB returned Count=3 and ScannedCount=3. Lambda logged successful writes at 02:21:29.811, 02:21:32.049, and 02:21:35.012 UTC on 27 September 2026. The approximately three-second spacing matches the configured simulator interval and demonstrates one successful processing event per publish.

B. Bidirectional control

The simulator log recorded COMMAND_RECEIVED for the expected device topic and payload containing command_id cmd-001 and command fan-on. This verifies that the design is not telemetry-only: the cloud can address the simulated actuator through a constrained topic.

C. Shadow convergence

The desired update produced a delta containing target_temperature 15. The simulator published a reported document with the same value. The final Shadow contained desired.target_temperature=15 and reported.target_temperature=15, while simulator_online=false accurately reflected the orderly disconnect. Thus cloud intent, device observation, and lifecycle state were consistent.

D. Authorization failure

The certificate was accepted for connection, isolating authentication from authorization. The later subscribe to the forbidden wildcard failed with an MQTT cancellation/hang-up response and the program emitted EXPECTED_AUTHORIZATION_FAILURE. This is positive evidence that the policy does not grant broad topic access.

E. Software and manifest validation

All eight FastAPI tests passed in 0.77 seconds. They cover legacy health and CPU routes plus telemetry JSON conversion, command publication, Shadow retrieval, and desired-state update. Kustomize rendered the Namespace, Service, version 2.0 Deployment, and autoscaling/v2 HPA without errors. Docker and Minikube daemons were unavailable in the final workstation session; therefore the HPA behavior reported for the extension relies on the unchanged manifest and the previously executed TA1 workload trials rather than claiming a new runtime trial.

IX. COMPARISON WITH THE INTERMEDIATE PROJECT

Dimension	Intermediate	Final extension
Inputs	HTTP CPU requests	HTTP plus MQTT
State	stateless API	DynamoDB and Shadow
Elasticity	HPA pods	HPA plus managed services
Identity	cluster workload	X.509 device identity
Observability	kubectrl/events	plus CloudWatch logs

IX. COMPARISON (CONT.)

The intermediate architecture was synchronous and intentionally stateless. Its primary concern was how Metrics Server and HPA transformed CPU utilization into replica changes. The extension adds asynchronous delivery, independent failure domains, durable state, per-device identity, and eventual reconciliation. These are new aspects not covered by the original assignment and directly satisfy option 2.2.

A. Elasticity

FastAPI remains bounded by one to six replicas under HPA. Lambda and DynamoDB scale through AWS-managed mechanisms, and IoT Core abstracts broker capacity. This reduces infrastructure management but distributes scaling control across providers and services. End-to-end capacity must therefore consider topic limits, Lambda concurrency, DynamoDB throttling, and API pods rather than treating HPA as the only controller.

B. Failure handling

The intermediate API returned a response or failed synchronously. MQTT QoS 1 may redeliver, Lambda may retry, and an offline device may receive state later through Shadow. The composite DynamoDB key provides simple idempotence for duplicate sequence numbers. A production design would add a dead-letter destination, conditional writes, expiry, alarms, and explicit command acknowledgments.

C. Operational complexity

Managed services remove broker installation and database capacity planning for the small workload, but IAM, certificate lifecycle, topic design, logging, and cleanup add operational work. The experiment required more distinct security boundaries than TA1. Automation and deterministic names reduced manual error and allowed resource-specific teardown.

X. COST ANALYSIS

The final IoT experiment generated three small messages, three Lambda invocations, three writes, several data-plane calls, and minimal logs. At this scale, metered usage is negligible and may fall within account free allowances. Unlike the TA1 EKS run, no continuously billed EKS control plane, EC2 worker group, or load balancer was created for the extension experiment.

Service	Usage in experiment	Cost characteristic
IoT Core	few MQTT messages	per message/connection
Lambda	3 short invocations	request and duration
DynamoDB	3 writes + queries	on-demand requests/storage
CloudWatch	small log stream	ingestion/storage
CloudShell	interactive session	no separate charge

Costs should be described as approximate because price, region, free-tier status, retention, and taxes vary. The dominant risk for this specific design is not the three-message experiment but leaving resources, log retention, or future traffic unattended. The cleanup script and post-experiment verification are therefore part of the cost-control design.

XI. VALIDITY AND LIMITATIONS

A three-message run proves integration, not throughput or long-term reliability. CloudShell and one simulated device do not represent unstable radio links, constrained microcontrollers, battery behavior, or a fleet. No end-to-end latency distribution was instrumented, and AWS timestamps show processing order rather than a calibrated network benchmark.

The absence of a final-session Docker/Minikube daemon prevented a fresh runtime HPA cycle for version 2.0. The API preserves the same /cpu implementation and the final overlay preserves the same 100m request, 50 percent target, and one-to-six limits. The repository is honest about this boundary and reuses the six real TA1 HPA trials only for architectural comparison, not as fabricated final-version measurements.

A. Internal validity

The strongest evidence is the agreement among independent services: the three simulator sequences equal the three Lambda success records and the three queried database items. The command and Shadow tests were executed during the same authenticated session. The negative test connected first and failed only when requesting the excluded topic, reducing the chance that a network or certificate error was misclassified as authorization enforcement.

B. External validity

The architecture generalizes to many telemetry applications, but the measured results should not be generalized to fleet scale. Broker quotas, Lambda concurrency, partition-key distribution, retry volume, and log ingestion become material with many devices. A single partition per device is suitable for ordered history but aggregate cross-device analytics would require secondary indexes, export, or a dedicated analytical store.

C. Reliability improvements

A production version should include command acknowledgments with correlation identifiers, conditional writes, a dead-letter destination, CloudWatch alarms, certificate rotation, fleet provisioning, and explicit retention. It should also distinguish transient sensor faults from connectivity loss and use timestamps assigned at both device and ingestion boundaries to estimate clock skew and transport delay.

XII. PRACTICAL LESSONS

Five practices were decisive: freeze the baseline before extending it; make resource names deterministic; test authorization failure rather than inferring least privilege from policy text; correlate one stable sequence across every processing stage; and postpone destructive cleanup until evidence and the rendered article are complete. These practices reduce both scientific ambiguity and operational risk.

CloudShell was useful because it reused the authenticated console session without creating long-lived IAM access keys on the workstation. Conversely, it reinforced that browser terminals are ephemeral: evidence must be exported or transcribed in sanitized form, and certificates must not become repository artifacts. Automation converts that temporary environment from a source of manual drift into a reproducible execution surface.

XIII. REPRODUCIBILITY

The public repository separates app, iot, aws, k8s, tests, evidence, documentation, and articles. The ta1-submission tag identifies the intermediate baseline, while the final-submission tag will identify the reviewed extension. Requirements are pinned, source and manifests are commented, and secrets are excluded by both directory practice and file patterns.

A reproducer installs the Python dependencies, runs pytest, builds the two images, and renders the final Kustomize overlay. In authenticated AWS CloudShell, aws/setup_final.sh provisions the backend. The simulator then executes run and forbidden-subscribe modes. AWS CLI queries validate DynamoDB, Shadow, and CloudWatch. After evidence and article review, aws/cleanup_final.sh removes the named resources and local certificate files.

XIV. CONCLUSION

This work successfully extended a Kubernetes HPA experiment into a complete software-only cloud IoT solution. All live AWS acceptance criteria passed: mutual-TLS MQTT connectivity, three telemetry messages, command delivery, rule-triggered Lambda processing, three DynamoDB items, Shadow convergence to 15, and denial of an unauthorized subscription. Eight automated API tests and a valid final Kustomize render complement the cloud evidence.

The central architectural lesson is that elasticity is layered. HPA scales stateless HTTP compute according to observed CPU, whereas AWS IoT Core, Lambda, and DynamoDB provide managed event and storage elasticity. Integration introduces durable state and device identity, but also topic authorization, certificate lifecycle, retry semantics, and cleanup responsibilities.

For this assignment, simulation is an effective substitute for hardware because it exercises the cloud control and data planes without weakening authentication or authorization. Future work should run a fleet of containerized simulators, measure latency and duplicate delivery, implement command acknowledgments, add alarms, and validate the final API image under a fresh HPA load cycle when a container runtime is available.

REFERENCES

- [1] OASIS, 'MQTT Version 5.0,' OASIS Standard, 2019.
- [2] Amazon Web Services, 'AWS IoT Core developer guide,' accessed Sep. 27, 2026. <https://docs.aws.amazon.com/iot/latest/developerguide/>
- [3] Amazon Web Services, 'AWS IoT policies,' accessed Sep. 27, 2026. <https://docs.aws.amazon.com/iot/latest/developerguide/iot-policies.html>
- [4] Amazon Web Services, 'AWS IoT Device Shadow service,' accessed Sep. 27, 2026. <https://docs.aws.amazon.com/iot/latest/developerguide/iot-device-shadows.html>
- [5] Amazon Web Services, 'Creating an AWS IoT rule,' accessed Sep. 27, 2026. <https://docs.aws.amazon.com/iot/latest/developerguide/iot-create-rule.html>
- [6] Amazon Web Services, 'AWS Lambda developer guide,' accessed Sep. 27, 2026. <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
- [7] Amazon Web Services, 'Amazon DynamoDB developer guide,' accessed Sep. 27, 2026. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>
- [8] Kubernetes Authors, 'Horizontal Pod Autoscaling,' accessed Sep. 27, 2026. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [9] Amazon Web Services, 'AWS IoT Core pricing,' accessed Sep. 27, 2026. <https://aws.amazon.com/iot-core/pricing/>
- [10] D. Bernstein, 'Containers and Cloud: From LXC to Docker to Kubernetes,' IEEE Cloud Computing, vol. 1, no. 3, pp. 81-84, 2014.

APPENDIX A. ACCEPTANCE MATRIX

Artifact	Result
Public source repository	Available
Intermediate immutable tag	ta1-submission
Commented automation	Included
Sanitized live evidence	Included
Cleanup automation	Included